Introduction

Along these tutorials, we will see how **SCOPE** interacts with the different parts of the code to handle the execution of computational workflows.

These are the topics covered in each tutorial:

1. The **System** class and its sources: the **Specie**, **Cell** and **Atom** classes
2. The Computational workflow: **Branch**, **Workflow**, **Job**, and **Computation** classes
3. The **State** class
4. The **Data**, **Collection** and **VNM** classes
5. The **Input_data** class, and **scope input files**
6. Running **SCOPE** - Part 1: File Structure
7. Running **SCOPE** - Part 2: Execution
8. Running **SCOPE** - Part 3: Detailed Actions
9. The Azo add-on: Subclasses, Analysis and Creation of Azo Systems

Tutorial 7: Running SCOPE, part 2

- In Tutorial 1, you saw the System class and its sources, and we created an example System, water.
- In Tutorial 5, you saw the Input_data class, and created a scope input file.
- In Tutorial 6, you saw (i) how to set up an Environment, (ii) how to submit a set of computational tasks, and (iii) how to manage the multiple files created and managed by **scope**.

Here, we will execute those tasks for water, and for two other simple systems.

In [1]:
```
import os
import scope
```

In [2]:
```
## Path of Tutorial 7's Data folder. Normally, it should be "os.path.abspath
tutorial_folder = os.path.abspath('../')+'/Data/7-Tutorial_7/'
```

1.1) Create Systems: Water, Benzene, Formaldehyde

Loading [MathJax]/extensions/Safe.js

In Tutorial 1, we created a System called "water", and imported a source, which was an .xyz file of a water geometry.

Here, we will load this System and complement it with 3 more structures, and we will also create two more Systems: benzene and formaldehyde

In [3]:
```
water_sys   = scope.load_binary(f"{tutorial_folder}Systems/water/water.npy")
benz_sys    = scope.System("benzene")
form_sys    = scope.System("formaldehyde")
```

In [4]:
```
## We set the paths for the two new systems. For water, we did it in Tutoria
benz_sys.set_main_path(tutorial_folder)
form_sys.set_main_path(tutorial_folder)

## We store the 3 systems in a list. At this stage, systems only have the pa
list_of_systems = [water_sys, benz_sys, form_sys]
print(list_of_systems)
```

Loading [MathJax]/extensions/Safe.js

```
[----------------------------------
   >>> SCOPE System Object >>>
----------------------------------
 Version               = 1.0
 Type                  = system
 Subtype               = system
 Source Path           = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Sources/water/
 System File Path      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/water/
 System File Name      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/water/water.npy
 Computations Path     = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Computations/water/

 Num Sources           = 4
     idx: type, name, formula
     0: specie, geom1, H2-O
     1: specie, geom2, H2-O
     2: specie, geom3, H2-O
     3: specie, geom4, H2-O

, ----------------------------------
   >>> SCOPE System Object >>>
----------------------------------
 Version               = 1.0
 Type                  = system
 Subtype               = system
 Source Path           = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Sources/benzene/
 System File Path      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/benzene/
 System File Name      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/benzene/benzene.npy
 Computations Path     = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Computations/benzene/

, ----------------------------------
   >>> SCOPE System Object >>>
----------------------------------
 Version               = 1.0
 Type                  = system
 Subtype               = system
 Source Path           = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Sources/formaldehyde/
 System File Path      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/formaldehyde/
 System File Name      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/formaldehyde/formaldehyde.npy
 Computations Path     = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Computations/formaldehyde/

]
```

Loading [MathJax]/extensions/Safe.js

1.2) Import Molecules in Sources Folder (sys.sources_path)

In [5]:
```
# Currently, SCOPE is designed so each system has its own folder with source

# Notice that there is no universal way of creating systems. Each problem in
# In the SCO add-on, the original data is in a cell2mol CELL format, so a ne
# New add-ons are being prepared, each one with its own form of importing sy

# For the moment, the simplest way to import sources is when those are in an
for tsys in list_of_systems:
    tsys.load_multiple_xyz(tsys.sources_path)
    tsys.save()
```

```
SYSTEM.ADD_SOURCE: Source with name 'geom1' already exists in system 'water'
SYSTEM.ADD_SOURCE: If you would like to Overwrite, specify overwrite=True
SYSTEM.ADD_SOURCE: Source with name 'geom2' already exists in system 'water'
SYSTEM.ADD_SOURCE: If you would like to Overwrite, specify overwrite=True
SYSTEM.ADD_SOURCE: Source with name 'geom3' already exists in system 'water'
SYSTEM.ADD_SOURCE: If you would like to Overwrite, specify overwrite=True
SYSTEM.ADD_SOURCE: Source with name 'geom4' already exists in system 'water'
SYSTEM.ADD_SOURCE: If you would like to Overwrite, specify overwrite=True
```

In [6]:
```
## When you run the cell above for the first time, you should receive a WARN
## This is because Systems do not accept new sources whose name is already u
## Remember in Tutorial 1 we already imported geom1.xyz

## Similarly, if you try to re-run the cell above several times, you'll rece
```

In [7]:
```
## We now added the .xyz files with molecules as sources. Our systems are re
print(list_of_systems)
```

Loading [MathJax]/extensions/Safe.js

```
[----------------------------------
   >>> SCOPE System Object >>>
----------------------------------
 Version               = 1.0
 Type                  = system
 Subtype               = system
 Source Path           = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Sources/water/
 System File Path      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/water/
 System File Name      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/water/water.npy
 Computations Path     = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Computations/water/

 Num Sources           = 4
     idx: type, name, formula
     0: specie, geom1, H2-O
     1: specie, geom2, H2-O
     2: specie, geom3, H2-O
     3: specie, geom4, H2-O

, ----------------------------------
   >>> SCOPE System Object >>>
----------------------------------
 Version               = 1.0
 Type                  = system
 Subtype               = system
 Source Path           = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Sources/benzene/
 System File Path      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/benzene/
 System File Name      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/benzene/benzene.npy
 Computations Path     = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Computations/benzene/

, ----------------------------------
   >>> SCOPE System Object >>>
----------------------------------
 Version               = 1.0
 Type                  = system
 Subtype               = system
 Source Path           = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Sources/formaldehyde/
 System File Path      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/formaldehyde/
 System File Name      = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Systems/formaldehyde/formaldehyde.npy
 Computations Path     = /Users/sergivela/Documents/SCOPE/Program/Scope_Tuto
rials/Data/7-Tutorial_7/Computations/formaldehyde/

]
```

Loading [MathJax]/extensions/Safe.js

2) Create SCOPE input files for tasks 2 and 3

In tutorial 5, we created the input for task1, which was a Single Point computation. Now we'll do the same for task2 and task3:

- Task2 will be a geometry optimization.
- Task3 will be a frequency computation.

Notice that, as opposed to Gaussian16, where you can specify both tasks in a single input file, in **SCOPE** these belong to different tasks.

In [8]:
```
## See Tutorial 5 for more information

task1_input = """
&environment
   max_jobs         = 24
   max_procs        = 96
   requested_procs  = 4
/

&options
/

&job_data
   branch       = 'tutorial'
   workflow     = 'all_sources'
   job          = 'opt'

   hierarchy    = 2
   requisites   = ['scf']
   constrains   = ['self']
   must_be_good = True

   istate       = 'initial'
   fstate       = 'pbe_opt'
/

&qc_data
   software     = 'g16'
   comp_type    = 'opt'
   functional   = 'pbe'
   basis        = 'sto-3g'
   is_grimme    = True
   loose_opt    = True
/"""
```

In [9]:
```
task2_input = """
&environment
   max_jobs         = 24
   max_procs        = 96
   requested_procs  = 4
/
```

Loading [MathJax]/extensions/Safe.js

```
&options
/

&job_data
   branch       = 'tutorial'
   workflow     = 'all_sources'
   job          = 'freq'

   hierarchy    = 3
   requisites   = ['opt']
   constrains   = ['self']
   must_be_good = True

   istate       = 'pbe_opt'
   fstate       = 'pbe_opt'
/

&qc_data
   software     = 'g16'
   comp_type    = 'freq'
   jobtype      = 'freq'
   functional   = 'pbe'
   basis        = 'sto-3g'
   is_grimme    = True
/"""
```

In [10]:
```
# IMPORTANT. Notice that, when running the 'scope_run_job' script. The job f
# Thus, we save this text to an actual file, so we can call it later
file_name1 = ''.join([tutorial_folder,"task2.scope"])
file_name2 = ''.join([tutorial_folder,"task3.scope"])
scope.read_write.save_text(task1_input, file_name1)
scope.read_write.save_text(task2_input, file_name2)
```

3) Run SCOPE

Once the system and a job_file are prepared, we can submit the computation in the appropriate cluster. To do so:

(1) Configure the Environment in the cluster by typing "scope config":

- The computation cluster must have Gaussian 16 installed
- You can give the environment any name, but for the sake of simplicity, here we will call it "scope_env_cluster.npy"

(2) Upload the system and scope input files to the cluster. In principle, you should upload the system files to dedicated folders inside the **env.systems_path** of the cluster enviromnent (env).

For instance, if env.sys_path = '/home/user/Scope_Tutorials/Data/7-Tutorial_7/2-

Loading [MathJax]/extensions/Safe.js

e system file should go to:

- '/home/user/Scope_Tutorials/Data/7-Tutorial_7/2-Systems/water/water.npy'
- '/home/user/Scope_Tutorials/Data/7-Tutorial_7/2-Systems/benzene/benzene.npy'
- '/home/user/Scope_Tutorials/Data/7-Tutorial_7/2-Systems/formaldehyde/formaldehyde.npy'

In practice, you can upload the system files anywhere in the cluster. However, the cluster's Environment will force the systems to be saved in their expected paths above. This is the minor price to pay for the Environment controlling the paths.

(3) Run: 'scope run'. If the paths are the expected ones, you should type:

```
scope run -n scope_env_cluster.npy -s water -t task1.scope
task2.scope task3.scope -e
scope run -n scope_env_cluster.npy -s benzene -t task1.scope
task2.scope task3.scope -e
scope run -n scope_env_cluster.npy -s formaldehyde -t task1.scope
task2.scope task3.scope -e
```

The **-e** option handles some errors that are relatively common with Gaussian16 or Quantum Espresso, at least in the computational clusters where I could test SCOPE. These 'software' errors could not be solved by the support teams of those clusters, so they seem to be inherent. They appear randomly, and are solved by re-submitting the job. Thus, whenever one of those errors is detected, SCOPE will just delete the aborted output and resubmit.

(4) Check and Wait.

- If everything worked well, then you should have gaussian16 jobs running in your cluster. Whenever one -or more- of those jobs finish, proceed to the next tutorial
- Otherwise, if all jobs fail or are aborted by SLURM/SGE, check the 'gen_g16_subfile' function in 'core/src/scope/software/g16_input.py'.
  This function creates the .sub files for G16 jobs. You might have to change it depending on how the computation cluster is set up.
  If you fail to make it work, please contact me at sergi.vela@iqac.csic.es

4) Larger Scale

Executing "**scope run**" for each system in the command interface is not feasible for a large number of systems.

Basically, scope run just calls the function "run_task" in scope.utils.
You can find below a python script that calls this function for all systems located by the environment.
This can be used as a template, and should facilitate the large scale submission of SCOPE tasks.

Loading [MathJax]/extensions/Safe.js

In [11]:
```
# import os
# import scope
# from scope import *
# from scope.utils.run_task import *
#
# ##### ENVIRONMENT #####
# env_path = "/path_to_environment_file"
# env = load_binary(env_path)
#
# #### INPUT ####
# debug           = 1            # 0 is equivalent to -q
# handle_errors   = True         # True is equivalent to -e
# ###############
#
# jobs = ['task1.scope','task2.scope','task3.scope']
#
### Option 1 ###
# report = ''
# for core in sorted(os.listdir(env.systems_path)):
#     path_to_system = env.systems_path+core+'/'+core+".npy"
#     report += run_task(path_to_system,jobs,env,handle_errors=handle_errors
# print(report)

### Option 2 ###
# report = ''
# systems = env.get_all_systems(debug=debug) ## Retrieves all systems in the
# for sys in systems:
#     report += run_task(sys.system_file,jobs,env,handle_errors=handle_error
# print(report)
```

In [ ]:

Loading [MathJax]/extensions/Safe.js